

Tema 2: Capa Acceso a Datos con Spring y JPA

1. Inyección de Dependencias

El patrón de Inyección de Dependencias (DI) permite que el contenedor de Spring (IoC) se encargue de instanciar y suministrar los objetos que una clase necesita para funcionar, reduciendo el acoplamiento.

- **Anotación `@Autowired`**: Se utiliza para inyectar automáticamente una dependencia. Por ejemplo, en nuestro `UserController`, inyectamos el `UserService` y el `JwtGenerator`.
- **Registro de Componentes**: Para que Spring pueda inyectar una clase, esta debe estar registrada en el contenedor. Esto se logra anotando la clase con el estereotipo adecuado, como `@Service` para la capa de negocio, o `@RestController` para los controladores REST. Si olvida anotar la implementación de un servicio con `@Service`, la aplicación fallará al intentar arrancar.

2. Transaccionalidad (`@Transactional`)

La transaccionalidad garantiza que un bloque de operaciones sobre la base de datos se ejecute como una unidad atómica. Se define mediante la anotación `@Transactional`, habitualmente a nivel de clase o método en la capa de servicios.

2.1. Reglas fundamentales de Examen

Este apartado es objeto frecuente de evaluación. Presten especial atención a los siguientes comportamientos por defecto de Spring:

- **Propagación**: Si la implementación de una operación invoca a otra operación de un servicio diferente (y ambas están anotadas con `@Transactional`), la segunda operación **se engancha a la transacción de la primera**. No se abren dos transacciones independientes.
- **Solo lectura (`readOnly = true`)**: Utilizada en métodos de consulta como `login`. **Atención**: Una operación anotada con `readOnly = true` **Sí se ejecuta dentro de una transacción**. Simplemente, informa al motor de persistencia de que puede optimizar el proceso al no requerir el volcado de modificaciones (*flush*).
- **Política de Rollback**: Si se lanza una excepción comprobada (Checked Exception*, como `InstanceNotFoundException`), **por defecto la transacción NO hace rollback**.
 - El *rollback* automático solo se produce ante excepciones no comprobadas (*Unchecked Exceptions*, que heredan de `RuntimeException`).

3. Optimistic Locking (Control de Concurrency Optimista)

En aplicaciones concurrentes (como la reserva de entradas de cine), varios usuarios podrían intentar adquirir el mismo recurso simultáneamente. Para evitar inconsistencias (por ejemplo, vender más asientos de los disponibles), utilizamos el bloqueo optimista.

- **Implementación:** Se define un atributo en la entidad, comúnmente numérico, y se le aplica la anotación `@Version` de JPA.
- **Funcionamiento:** En la entidad `Session`, tenemos un campo `Long version`. Al realizar un `UPDATE`, Hibernate comprueba que la versión en memoria coincide con la de la base de datos. Si coinciden, actualiza los datos e incrementa la versión. Si no coinciden (porque otra transacción modificó la fila en el ínterin), se lanza una excepción de concurrencia y se aborta la transacción, protegiendo así la integridad de los asientos libres.

4. Pruebas Automatizadas (Testing)

Las pruebas de integración en el backend se apoyan en un entorno de Spring levantado específicamente para ello.

4.1. Configuración del entorno de pruebas

- **Aislamiento:** Empleamos la anotación `@ActiveProfiles("test")`. Esto indica a Spring que aplique la configuración específica para pruebas (por ejemplo, contenida en un archivo `application-test.yml`), sobreescribiendo la configuración por defecto de producción.
- **Limpieza Automática:** Las clases de prueba (`@SpringBootTest`) suelen anotarse con `@Transactional`. Esto altera el comportamiento normal: al finalizar cada método `@Test`, la transacción realiza un **rollback automáticamente**. Por lo tanto, el desarrollador **no debe limpiar manualmente** la base de datos (por ejemplo, ejecutando `userDao.delete(...)`).

4.2. Identidad de los Objetos (Caché de Primer Nivel)

- **Pregunta clásica de examen:** En un test que crea un objeto `User`, lo guarda en la base de datos y luego lo recupera por su ID dentro de la misma prueba, **no es necesario redefinir los métodos `equals` y `hashCode`** en la entidad para que una aserción como `assertEquals(user, user2)` funcione correctamente.
- **Justificación:** Al estar todo el código de la prueba enmarcado dentro de una única transacción, la sesión de Hibernate (caché de primer nivel) devuelve la misma referencia de memoria para el objeto persistido y el recuperado.